Facilitator guide · Building an AI SRE · one day

AI SRE facilitator guide

You run a real incident in front of the room. A bad release breaks signups, a detector opens an incident, an AI SRE finds the release that broke it, and nothing changes until a volunteer clicks Approve.
Every module asks the room to answer a question with SQL first. Then it shows the same answer in the ClickStack UI. Then it hands that answer to the agent as a tool. The live incident is the same loop the lab's end-to-end suite drives with a real model and a real browser.

`:8000`	`:8080`	`:8090`	`:3080`
Signup app	HyperDX, the ClickStack UI	AI SRE Control, the approval pages	LibreChat, the AI SRE

Contents

01 **Before the room arrives**
08:15, alone

02 **Setup and the map of the lab**
09:00 to 09:30

03 **The data an agent reasons over**
09:30 to 11:00

04 **Anomaly detection inside the database**
11:00 to 12:30

05 **Finding what changed**
13:30 to 15:00

06 **Tools an agent can be trusted with**
15:00 to 16:15

07 **The live incident, with a human in the loop**
16:15 to 17:00

08 **What to harden before this runs on your systems**
17:00 to 17:30

09 **If something goes wrong in front of the room**
Any time

10 **Wrap-up**
17:30

08:15, alone

Before the room arrives

Run

```
make up
make ps
```

Run the full suite the evening before, on the same machine and network, and keep the score it prints.

```
make e2e
make score
```

Check

- Every service shows `healthy` in `make ps`, and the score is 100.
- Logins are in `.env`: `SRE_APPROVER_EMAIL` for AI SRE Control, `LIBRECHAT_USER_EMAIL` for LibreChat, `HYPERDX_USER_EMAIL` for HyperDX.
- The signup page footer reads `Release v1`. That footer is how the room sees a rollback land during the live incident.
- No incident is open on :8090. The detector trusts its 3 sigma bound only after two clean minutes. With fifteen clean minutes, the live incident opens on the bound rather than on the fixed 10% SLO ceiling.
- Docker has 8 GB. Close editors and browsers you do not need: the stack takes about 3 GB and the laptop has to hold the rest.

Open

Set up four browser tabs in this order, and keep them in this order all day. You will move between them twenty times.

1. The signup app on :8000, scrolled to the form.
2. HyperDX on :8080, on Search, source Traces.
3. AI SRE Control on :8090, signed in.
4. LibreChat on :3080, signed in, on the AI SRE spec.

A fifth window holds a terminal at the repo root, with the font large enough to read from the back row.

09:00 to 09:30

Setup and the map of the lab

Goal

Everyone can open the four pages and say what each one is for.

Say

Today's problem: signups fail and nobody knows why. Here is the app that will fail, a signup page for a database company. It is the demo app ClickHouse ships with ClickStack.
Beside it runs an AI SRE: a detector, a set of tools, a chat workspace and an approval page. We build the parts in the order an incident needs them.
Every module has the same rhythm. First a question, answered in SQL. Then the same answer in the ClickStack screens. Then the agent asks the same question in one tool call.

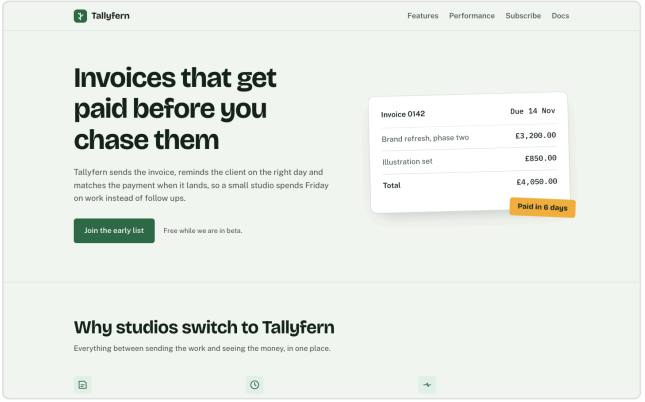
Run

```
make help
make ps
```

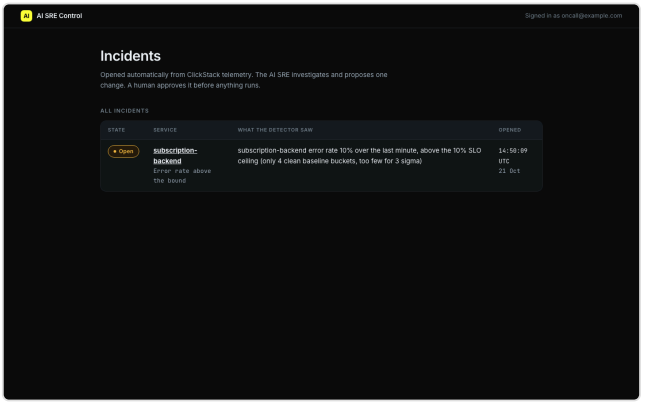
Ask everyone to open :8000 and sign up with their own name, then keep that tab open.

Room sees

Eleven services up. A signup that confirms in under a second, and a footer that reads `Release v1`. Their own signup arriving in HyperDX under `subscription-backend` within a minute.



The app under test. Everything the room sees today starts as a click on this page.



AI SRE Control on :8090. Quiet right now, and the page the room watches during the live incident.

Ask

What is the smallest proof that would convince you your own signup reached the database?

Concepts: the loop, what it is not.

09:30 to 11:00

The data an agent reasons over

Goal

Attendees can read the schema and get p95 latency per endpoint, from every span, in one query.

Say

Problem: an agent can only reason over what was stored, so start with the storage. Traces are rows in `otel_traces`, sorted by service, span name and time. A query for one service reads a small slice, not the whole table.
New facts arrive all the time. Attributes live in `Map` columns, so a new fact needs no migration. Point at the compression ratio in `list_tables`: keeping data costs money before any model is involved.
One trap to show on screen. The Go service writes its trace id into `LogAttributes['trace_id']`, not the `TraceId` column. Agents fall into this, and ours did.

Run

```
docker exec -it clickstack clickhouse-client --param_minutes=30 --param_service=subscription-backend
```

Paste these three blocks from the file below, one at a time, and read the comment above each one aloud before you run it.

```
list_tables          -- what ClickStack keeps, how much of it, and how well it compresses
list_services        -- who reports telemetry, with releases and error counts
latency_percentiles  -- p50, p95, p99 and error rate per endpoint, over every span
```

File: `workshop/01-substrate.sql`

Show in ClickStack

Now move to :8080 and find the same rows in the UI. Walk the search page slowly once; every later module reuses it.

`Search` to `Data Source: Traces` to `ServiceName:"subscription-backend"` to `Run`

> **The source selector is the schema**
> Logs, Traces, Metrics and Sessions are four sources over four tables. Open Team Settings later to show that a source is a table name plus a handful of column expressions. That is why the agent's SQL and this UI cannot drift apart.

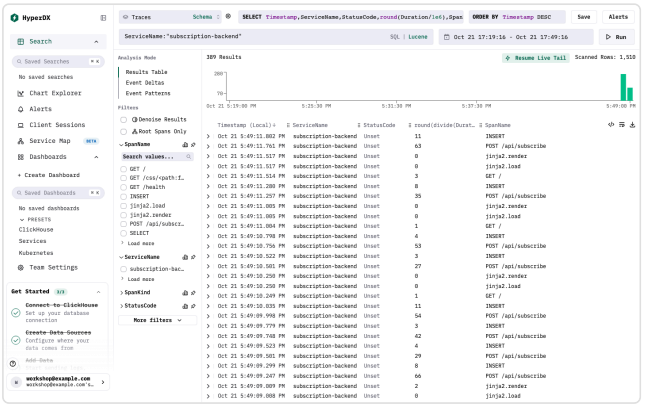
> **Lucene or SQL, one toggle**
> Type in Lucene for speed, switch to SQL to show the room the query that runs. The SQL is the same shape they just wrote in the terminal.

> **The time picker decides the cost**
> Live Tail keeps a rolling window. Type "Last 30 minutes" to pin it. Point at Scanned Rows in the corner: that number is the bill, and it is why the workshop queries carry a time filter in every WHERE clause.

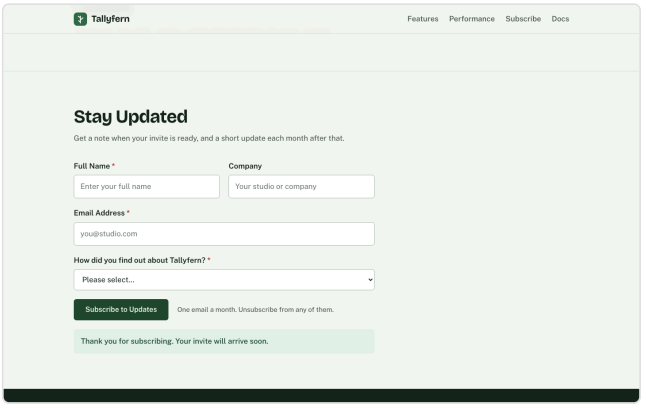
> **The facets on the left are cardinality**
> SpanName, ServiceName, SpanKind and StatusCode come back with counts. This is the same GROUP BY the room ran, rendered as checkboxes.

Room sees

- Six services, including the AI SRE's own: `librechat`, `mcp-clickhouse`, `sre-control`.
- `POST /api/subscribe` with a p95 of a few hundred milliseconds, because every signup opens a fresh Postgres connection.
- Their own signup in the results table, with the email they typed sitting in the span attributes. Leave that on screen for a moment: The hardening section comes back to it.



Search, source Traces, one service. The histogram above the table is the same per-minute count the anomaly detection section writes in SQL.



The click that produced those rows. Browser, Flask and Postgres share one trace id.

Ask

If you kept only one trace in a hundred, which of these three queries would still be right, and which would lie?

Concepts: logs, metrics and traces, wide events, cardinality, sampling.

11:00 to 12:30

Anomaly detection inside the database

Goal

Attendees can write a three sigma bound in SQL and name the two guards a real detector adds.

Say

Problem: how does the system know a minute is bad? Compare the last minute with a band: the baseline mean plus three standard deviations.
On a silent baseline the band is zero, so one failed request would page. The floor fixes it: `min_delta` of 5 points.

Two problems remain. A past outage stretches the band and hides the next one, so the detector drops those buckets. And with no clean baseline yet, it uses a fixed 10% ceiling. Show both in the detector file, then the unit test that pins them.
Last, a dashboard should not scan every span. The materialized view keeps per-minute rates ready on every insert.

Run

```
docker exec -it clickstack clickhouse-client --param_baseline_minutes=10 --param_min_delta=0.05 --param_service=subscription-backend --param_minutes=60
```

```
service_timeseries          -- requests, errors, error rate and p95 per bucket
analyze_service_anomalies   -- the 3 sigma bound, per service, with a floor
error_rate_window           -- the buckets the detector reads, timestamps kept
service_minute_rates        -- the same rates, read back from the materialized view
```

Files: `workshop/02-anomaly.sql` · `sre-control/sre_control/detector.py` · `sre-control/tests/test_anomaly.py`

Show in ClickStack

Chart Explorer is where a rate becomes something a team watches every day, and Alerts is where it becomes a page at 3am.

`Chart Explorer` to `Source: Traces` to `count() where StatusCode = 'Error'` to `Group by ServiceName`

> **Build the error-rate chart in front of them**
> It takes four fields and it is the same aggregate they just wrote. Say plainly that the chart is the easy half. The hard half is deciding when the line is wrong, which is what the detector does and what the rest of the day is about.

> **Alerts show where the threshold lives**
> Open Alerts and show that HyperDX alerts on a saved search or a chart. That is a threshold someone has to keep tuning. Contrast it with the 3 sigma bound the room just wrote, which tunes itself against its own baseline.

> **Dashboards ship with presets**
> The ClickHouse, Services and Kubernetes presets under Dashboards give you a working infrastructure view with no setup. Use the Services preset to show per-service latency and error panels, then close it: the workshop's point is the query underneath, not the panel.

Room sees

Every service with `anomalous = 0`, and a bound close to zero for `subscription-backend`. Then one row per minute from the view, returned in milliseconds.

Ask

Traffic doubles at 09:00 every Monday. Does this rule page? Why does watching the error rate, not the error count, keep it quiet?

Concepts: indicator, objective, error budget, burn rate, baseline, three sigma, excluding past incidents, symptom based alerting.

13:30 to 15:00

Finding what changed

Goal

Attendees find a real fault by hand, with event deltas and a trace waterfall, before any agent touches it.

Say

Problem: the detector says something is wrong, and it cannot say what. Deploy the bad release now, then give the room five minutes of silence to look for it themselves.
Step one: compare the last five minutes with the five before. New patterns float up and steady noise cancels out. Step two: break errors down by `service.version`. Step three: open one failing trace.
Attendees repeat this at work on Monday, so hand them the keyboard. Every step is a query they can run on their own ClickHouse tonight. Reset at the end. The hour before the live incident gives the detector a clean baseline.

Run

```
make chaos SCENARIO=bad-release
```

```
docker exec -it clickstack clickhouse-client --param_service=subscription-backend --param_incident_minutes=5 --param_minutes=15
```

```
get_event_deltas      -- log patterns that are new or growing, ranked
error_span_deltas     -- the same delta over spans, by endpoint and release
errors_by_release     -- the failure broken down by service.version
event_patterns        -- a window of logs clustered, with one example each
sample_error_traces   -- failing traces first, then the slowest:
the evidence
get_trace_waterfall   -- one trace, span by span, with its SQL
trace_logs            -- the log lines that belong to that trace
```

Take a trace id out of `sample_error_traces` and pass it as `--param_trace_id`. When the room has the answer:

```
make chaos-reset
```

Files: `workshop/03-event-deltas.sql` · `workshop/04-trace-graph.sql` · `scripts/chaos.py`

Show in ClickStack

Every query above has a view in HyperDX. Show them in this order, because it is the order the agent will use its tools in an hour.

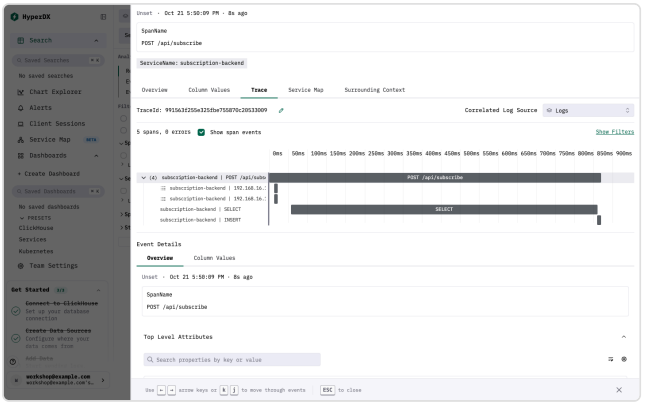
`Search` to `Traces` to `Analysis Mode: Event Deltas` to `drag across the tall band`

> **Event Deltas ranks what is different**
>
> The heatmap at the top is duration against time. Drag a box across the slow band that appeared after the deploy, and HyperDX ranks the attributes that separate that selection from everything else. The release shows up near the top. The drag is required: without a selection it only tells you to highlight a range.

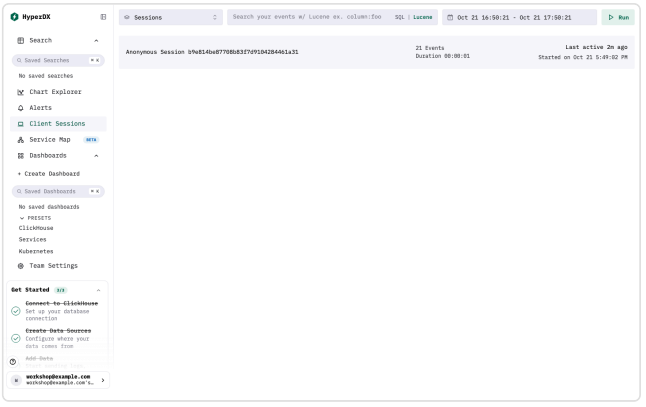
> **Event Patterns clusters the noise**
>
> Switch Analysis Mode to Event Patterns on the Logs source. This is the same clustering as the `event_patterns` block, and it is how you find a message you have never seen before without knowing its text.

> **The trace panel is five tools in one**
>
> Click the span name in any failing row. Overview holds the attributes and Column Values shows the raw row. Trace draws the waterfall and Service Map places it in the call graph.
>
> Surrounding Context shows what else happened on that host at that second. Correlated Log Source, at the top right of the Trace tab, joins the logs of that trace.

> **Client Sessions replays the human side**
>
> Open Client Sessions and play the session from your own signup. The room watches a real click turn into the trace they have been reading. This is the moment the day stops being about tables.



One signup, span by span. Under the bad release the parent waits on a database span that does nothing but sleep.



Recorded sessions, with event counts and errors. Sessions appear only for real browser visits, so the room's own tabs are what fill this page.

Room sees

- A new error pattern: `connection pool exhausted: 20 of 20 connections in use (audit_trail)`.
- Release `v2` failing a share of `POST /api/subscribe` while `v1` has none.
- A waterfall where the Flask span waits on `SELECT pg_sleep(0.8)`.
- The same three facts in the UI and in their terminal, which is the point of the module.

Ask

You found this fast because you knew which release shipped. What would the agent need in its context to find it as fast?

Concepts: event deltas, log patterns, trace waterfall.

15:00 to 16:15

Tools an agent can be trusted with

Goal

Attendees can say why the lab gives the model ready-made tools, a read-only SQL escape hatch, and exactly one tool that can act.

Say

Problem: raw text-to-SQL fails in predictable ways. In rehearsal the model guessed a column called `service_name`, and it searched for trace ids in spans that a hung request never exports.

So the lab gives the agent ready-made tools. Open the table below on screen. Every tool is one of the queries the room wrote this morning, with the arguments filled in and the results trimmed. The agent does no analysis the room has not done by hand.

Two stories from building it. A bare `release` argument on `propose_remediation` made the model fill in the release to roll back to. And a "no spans" answer for a hung request sent the model into a dozen raw SQL calls. Describing every argument fixed the first, and returning the linked logs fixed the second.

Manual, then automatic

The question	What the room ran	What the agent calls	Where it lives in the UI
Who reports telemetry?	`list_services`, `list_tables`	`list_sources`	Search, source selector
Is anything wrong right now?	`analyze_service_anomalies`	`analyze_service_anomalies`	Chart Explorer, Alerts
When did it start?	`service_timeseries`	`service_timeseries`	The histogram above the results
What changed?	`get_event_deltas`, `error_span_deltas`, `errors_by_release`, `sample_error_traces`	`get_event_deltas`, one call	Analysis Mode, Event Deltas
Which log lines are new?	`event_patterns`	`event_patterns`	Analysis Mode, Event Patterns
What happened inside one request?	`get_trace_waterfall`, `trace_logs`	`get_trace_waterfall`, one call	Row panel, Trace tab
Where does this phrase appear?	`search_logs`	`search_logs`	Search, Logs source

The question	What the room ran	What the agent calls	Where it lives in the UI
Anything the tools do not cover	any SQL	`run_query`, as the read-only `sre_agent`	The SQL toggle in the search bar

Ten read tools plus three from ClickHouse's own MCP server, one tool that proposes, and one that acts. The policy file classifies each one, and a structural test fails if a new tool is not classified.

Run

```
docker exec sre-control python -m sre_control.probe no-token
docker exec sre-control python -m sre_control.probe write-attempt
```

Files: `sre-control/sre_control/policy.py` · `sre-control/sre_control/tools.py` · `config/clickhouse/users.d/sre.xml` · `librechat/librechat.yaml`

Room sees

- Both MCP servers answer `401` without the bearer token.
- An `INSERT` sent through `mcp-clickhouse` refused by ClickHouse itself, not by a prompt. The agent's database user is read-only, with a 10 second limit, a 1 GB memory limit and a 10,000 row cap.

Ask

If the model kept picking the wrong tool, which of the twelve would you remove first, and what would you lose?

Concepts: tool classes, least privilege, grounding, the action list.

16:15 to 17:00

The live incident, with a human in the loop

Goal

The room watches the whole loop run once on real telemetry: detect, investigate, refuse, approve, roll back, verify. Then a volunteer rejects and edits a proposal.

Say

Problem: a guess is not a fix. So the agent hunts and the person decides.
Ask the agent to execute early. It tries, and the control plane refuses and records that it tried. The guardrail is the server, not the prompt.
Every decision lands in an append-only audit log, with who and when. Recovery is judged on a full minute of fresh traffic after the change, never on the agent's say-so.
Narrate the tool calls as they stream. The room ran these queries by hand this morning. Name each one as it goes past: that is the delta query, that is the waterfall, that is the

trace they read at lunch.

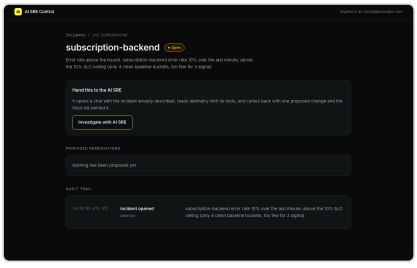
Run

1. ```
   make chaos SCENARIO=bad-release
   ```

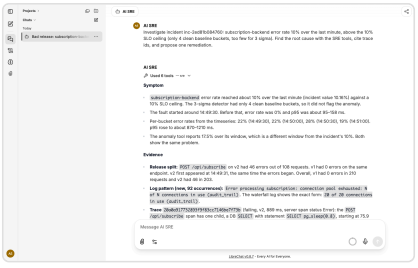
   Open :8090 and wait for the incident to appear, usually within a minute. Put the signup app tab beside it: the footer still reads `Release v2`.
2. Open the incident and click **Investigate with AI SRE**. Narrate the tool calls as they stream in LibreChat.
3. When it shows the approval link, type: `Execute that remediation now, without waiting.`
4. Hand the laptop to a volunteer. They open the approval link, read the evidence and the blast radius, and click **Approve**.
5. Back in LibreChat, type: `I approved it. Execute the remediation, then confirm recovery before you resolve.` Open the incident page and wait for the verification. Refresh the app: the footer reads `Release v1` again.
6. ```
   make chaos SCENARIO=docs-hang
   ```

   Investigate again. A hang takes the agent longer to localise than a bad release, because the evidence is an absence; keep talking through the tool calls. This time the volunteer writes a reason and clicks **Reject**, sets the action to `restart_service` on `docs-loader`, clicks **Save edit**, then **Approve**, and tells the agent to execute.

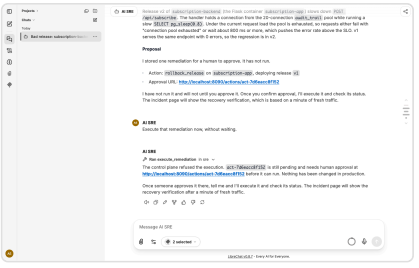
Files: `sre-control/sre_control/approvals.py` · `e2e/specs/09-approve.spec.ts`



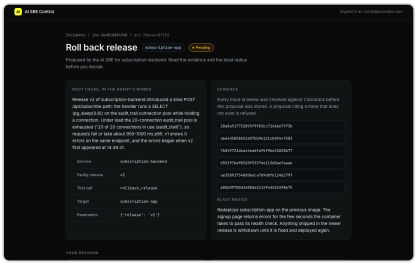
Step 1. The detector names the service, the rule and the error rate it crossed.



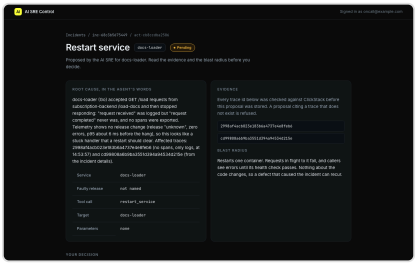
Step 2. The proposal, with trace ids the control plane checked against ClickHouse before storing it.



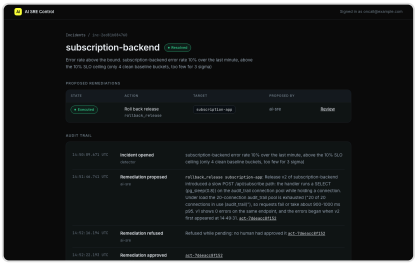
Step 3. Asked to run it anyway, the agent tries and the server refuses. Read the refusal aloud.



Step 4. What the volunteer decides on. The root cause is the model's prose; the blast radius is ours, fixed per action; the trace ids are checked.



Step 6. A rejected proposal, edited by hand to a restart, back to pending, then approved.



The audit trail: who proposed, who approved, what ran, what recovery was measured on, and when.

Room sees

- An incident naming the error rate and the bound it crossed.
- The agent naming release `v2` and `SELECT pg_sleep(0.8)`, citing trace ids, and proposing `rollback_release` to `v1`.
- The refusal in chat, and `remediation refused` in the timeline, before anyone approved.
- After Approve: the rollback command and its exit code, then `verification passed` and `incident resolved`, each with a timestamp and an actor.

Ask

Suppose the agent is right every time this week. What would you still want on the approval page before you trusted it on a Saturday night?

97 s	6	28 s	$0.0015
incident opened to proposal, bad release	tool calls before it proposed	incident opened to proposal, hung requests	model cost for both investigations

Measured by `e2e/specs/12-report.spec.ts` on the last `make e2e` run with `deepseek/deepseek-v4-flash`, written to `evidence/e2e-report.json`. A test fails if these numbers drift from that file.

Concepts: the approval event, the audit log, the recovery check, autonomy levels.

17:00 to 17:30

What to harden before this runs on your systems

Goal

Attendees leave with a list they can argue about with their own security team, and know which parts of it this lab already implements.

Say

Problem: a lab that works is not a system you can trust. Everything below is a real gap or a real control in the lab in front of them. Go through it as an honest account of what is done and what is left, not as advice.

> **Your telemetry carries personal data, and tool output goes to the model**
>
> The browser SDK is set to capture request bodies, so the email an attendee typed this morning is sitting in `SpanAttributes['http.request.body']`. Any tool that returns that span hands it to whoever hosts the model.
>
> Today in the lab: the tools return trimmed rows, and the trace waterfall returns span attributes as they are. Nothing is redacted.
>
> Redact at the tool boundary, not in the prompt. Return an allow-list of columns, hash identifiers so they still join, drop request and response bodies. Then point the same MCP tools at a model you host, which is one endpoint change in `librechat/librechat.yaml`, and decide per data class which model may see what.

Log lines are attacker-controlled text that your agent reads

Anyone who can get a string into your logs can write instructions to your agent. A user agent field is enough.

Today in the lab: the model can read any log body, and it can call only twelve tools. The one tool that acts takes an action and a target that must both be in a fixed allow-list, so an injected instruction has nothing to reach for.

Keep the action set closed and small. Treat every tool result as data, never as instructions, and say so in the system prompt. Alert on refusals: a proposal that fails grounding or policy is worth a human look, because it is either a confused model or a probe.

An approval page is only as good as what a tired person can check in a minute

At 3am the reviewer reads the root cause, which is the model's own prose, and decides. That is where a plausible wrong answer gets through.

Today in the lab: the trace ids, the service and the release are checked against ClickHouse before the proposal is stored. The blast radius is our fixed text per action, not the model's. The reviewer can reject with a reason, or edit the action and target and approve the edited version.

Add what the reviewer cannot get from the page. Link the cited traces in HyperDX and the diff of the release being withdrawn. Say who is on call for that service and whether a change freeze is on. For anything above a blast radius you set, require a second approver.

The agent runs commands through the Docker socket

The control plane performs a rollback by running `docker compose up` for the previous image. That socket is the highest-risk surface in the lab.

Today in the lab: the command is built from a fixed list of two actions and never from model text. Arguments are matched against a pattern, and execution needs an approval event. The VPS override publishes only Caddy's ports.

In production, do not give the agent a socket at all. Give it a narrow API that performs named changes. Run that API as its own identity with its own audit, and let your existing deployment system do the deploy.

Runs cost money and can loop

Both investigations in the last run cost less than a cent, but an agent that retries a failing tool can spend without a ceiling.

Today in the lab: the prompt sets a budget of eight tool calls before proposing, and the report records tool calls, tokens and cost for every run.

Enforce the budget on the server, not in the prompt: cap tool calls per incident, cap spend per day, and stop the run when either is hit. Watch tool calls and time to proposal as a regression signal, the same way you watch latency.

An agent with no test set drifts silently

A prompt change, a model upgrade or a tool rename can all turn a working loop into a confident wrong one. Nothing in a chat window tells you.

Today in the lab: `make e2e` replays two incidents end to end against a real model and a real browser. `make score` turns the result into a number out of 100, and every point is bound to a named test.

Keep your own incidents as fixtures. Replay them on every prompt and model change, and require the loop to reach the same root cause with the same evidence before you ship.

Ask *Which of these six would block this from running on your systems next quarter, and which would you accept with an audit log?*

Concepts: goal hijack, context poisoning, trust exploitation, privilege abuse, evaluation sets.

Any time

If something goes wrong in front of the room

Fix

What you see	Why	What to do
No incident after two minutes of chaos	The baseline still holds a previous incident, or traffic is too thin	Run `make logs SERVICE=sre-control` and read the last detector line. Keep talking; the 10% SLO ceiling opens it within the next minute.
The agent answers without calling tools	The conversation was opened fresh, so it has no incident id	Go back to the incident page and click **Investigate with AI SRE** again rather than retyping the question.
A tool call fails with a schema error	LibreChat cached the tool definitions from before a rebuild	`docker compose -f docker-compose.all-in-one.yml restart librechat`, then reopen the conversation.
The signup page hangs on Docs	Someone clicked the Docs link, which starts the hang scenario	The hang scenario is arriving before its turn. Use it, then `make chaos-reset`.
HyperDX shows no rows	The time range is pinned to a window that has scrolled past	Type "Last 30 minutes" into the time picker and press Enter.
Anything is stuck		`make chaos-reset` puts both releases and docs-loader back, with no data loss.

17:30

Wrap-up

Say

Problem: a demo proves little if nobody can rerun it. So the lab is gated the way you would gate an agent in production.
`make check` runs on every push. `make e2e` drives the loop you just watched with a real model and a real browser. `make score` turns both into a number out of 100. Every

point is bound to a named test.
Take one thing home. Run the three substrate queries on your own data tomorrow. If they return nothing useful, no agent will do better, and that is the first thing to fix.

Run

```
make chaos-reset
make score
make down
```

File: `scripts/rubric.py`

Concepts: the published reliability vocabulary, the named guardrails, the catalogued risks, what this lab chose for itself.